

Sistemi Operativi – a.a. 2025/2026

prova di laboratorio

– 20 maggio 2026 –

Creare un programma `balanced-vector-generator.c` in linguaggio C che accetti invocazioni sulla riga di comando del tipo:

`balanced-vector-generator <N> <M>`

Il programma si occuperà di generare N vettori “bilanciati” con M elementi (con M pari) con un processo casuale per fasi: un vettore di interi si dice “bilanciato” se il numero di elementi pari coincide con il numero di elementi dispari (quindi esattamente $M/2$ elementi pari e $M/2$ elementi dispari). Esempio:

66	3	40	88	21	7	12	9
----	---	----	----	----	---	----	---

4 elementi pari (66, 40, 88, 12) e 4 elementi dispari (3, 21, 7, 9); le posizioni dei numeri pari e dispari sono ininfluenti

In particolare al suo avvio il programma creerà 6 thread ausiliari:

- **un thread Generatore** che si occuperà di generare esattamente N vettori con M interi casuali (compresi tra 0 e 99), tale vettore sarà inserito in un record `<vettore,round>`, dove `round` è posto inizialmente a 0; il vettore sarà inserito in una coda proposte di record con vettori da verificare; dopo questo il thread terminerà;
- **2 thread Verificatore** che si occuperanno, in modo concorrente, di estrarre i record con i vettori da verificare dalla coda proposte; i vettori che soddisfano la proprietà saranno accettati e conteggiati; gli altri saranno inseriti in una coda scartati che conterrà record con vettori che dovranno essere “aggiustati” per essere poi rivalutati; una volta raggiunti i richiesti N vettori bilanciati, questi thread dovranno terminare;
- **3 thread Riparatore** che si occuperanno, in modo concorrente, di estrarre record dalla coda scartati processandoli nel seguente modo:
 - incremento del campo `round` nel record;
 - selezione casuale di uno degli M slot del vettore;
 - sostituzione con un altro numero casuale (sempre compreso tra 0 e 99) dello stesso.

Le strutture dati condivise saranno:

- una coda proposte di record con capienza massima di 15 elementi che conterrà i vettori da valutare;
- una coda scartati di record con capienza massima di 15 elementi che conterrà i record che necessitano aggiustamenti per poi essere rivalutati;
- mutex e semafori numerici: il loro numero, comunque minimale, e le modalità d’uso sono da determinare da parte dello studente;
- eventuali flag/contatori di fine-lavoro.

Il thread principale/main dovrà attendere la terminazione di tutti i thread ausiliari che dovranno terminare spontaneamente alla fine del loro compito. Non si dovranno usare strutture dati con visibilità globale. E’ necessario rispettare fedelmente la struttura dell’output riportato nell’esempio a seguire. Codici sorgente con errori che bloccano la compilazione e pregiudicano la generazione del codice binario con compilatori standard (gcc o clang) non saranno valutati.

Tempo: 2 ore e 30 minuti

L'output atteso di una possibile esecuzione è il seguente:

```
$ ./balanced-vector-generator 10 8
[MAIN] creazione di un thread generator, di 2 thread verificatore e di 3 thread
riparatori
[GEN] vettore candidato n.1: 76, 59, 92, 40, 21, 72, 90, 1
[GEN] vettore candidato n.2: 96, 80, 31, 46, 97, 24, 34, 99
[GEN] vettore candidato n.3: 43, 48, 12, 38, 37, 51, 8, 42
[GEN] vettore candidato n.4: 20, 37, 19, 58, 40, 50, 21, 16
[VER-2] estratto un vettore candidato con round pari a 0: 76, 59, 92, 40, 21, 72, 90, 1
[VER-2] vettore non verificato e rigettato (5 pari vs. 3 dispari)
[VER-2] estratto un vettore candidato con round pari a 0: 96, 80, 31, 46, 97, 24, 34, 99
[VER-2] vettore non verificato e rigettato (5 pari vs. 3 dispari)
[VER-2] estratto un vettore candidato con round pari a 0: 43, 48, 12, 38, 37, 51, 8, 42
[VER-2] vettore non verificato e rigettato (5 pari vs. 3 dispari)
[VER-1] estratto un vettore candidato con round pari a 0: 20, 37, 19, 58, 40, 50, 21, 16
[VER-1] vettore non verificato e rigettato (5 pari vs. 3 dispari)
[GEN] vettore candidato n.5: 10, 65, 57, 83, 38, 47, 84, 86
[REP-1] estratto un vettore da riparare: 76, 59, 92, 40, 21, 72, 90, 1
[GEN] vettore candidato n.6: 79, 15, 32, 77, 91, 18, 28, 86
[REP-1] reinserito vettore riparato (40->41) con round pari a 1: 76, 59, 92, 41, 21, 72,
90, 1
[REP-1] estratto un vettore da riparare: 96, 80, 31, 46, 97, 24, 34, 99
[REP-1] reinserito vettore riparato (34->33) con round pari a 1: 96, 80, 31, 46, 97, 24,
33, 99
[REP-3] estratto un vettore da riparare: 43, 48, 12, 38, 37, 51, 8, 42
...
[VER-1] estratto un vettore candidato con round pari a 1: 76, 59, 92, 41, 21, 72, 90, 1
[VER-1] vettore verificato e accettato dopo 1 round di aggiustamenti (1 di 10)
[VER-2] estratto un vettore candidato con round pari a 1: 96, 80, 31, 46, 97, 24, 33, 99
[VER-2] vettore verificato e accettato dopo 1 round di aggiustamenti (2 di 10)
[REP-2] estratto un vettore da riparare: 10, 65, 57, 83, 38, 47, 84, 86
[REP-2] reinserito vettore riparato (38->39) con round pari a 1: 10, 65, 57, 83, 39, 47,
84, 86
[VER-1] estratto un vettore candidato con round pari a 1: 10, 65, 57, 83, 39, 47, 84, 86
[VER-1] vettore non verificato e rigettato (3 pari vs. 5 dispari)
...
[REP-2] estratto un vettore da riparare: 6, 70, 33, 40, 36, 47, 64, 91
[REP-2] reinserito vettore riparato (70->9) con round pari a 27: 6, 9, 33, 40, 36, 47,
64, 91
[VER-1] estratto un vettore candidato con round pari a 27: 6, 9, 33, 40, 36, 47, 64, 91
[VER-1] vettore verificato e accettato dopo 27 round di aggiustamenti (9 di 10)
[REP-3] estratto un vettore da riparare: 12, 39, 30, 40, 55, 43, 10, 32
[REP-3] reinserito vettore riparato (40->41) con round pari a 31: 12, 39, 30, 41, 55, 43,
10, 32
[VER-2] estratto un vettore candidato con round pari a 31: 12, 39, 30, 41, 55, 43, 10, 32
[VER-2] vettore verificato e accettato dopo 31 round di aggiustamenti (10 di 10)
[VER] terminato con 10 vettori bilanciati trovati
[REP-3] terminato
[REP-2] terminato
[REP-1] terminato
[MAIN] terminato
```